Student Name: Juhi Surin
Register Number: 192525148
Course: CSA0919 – Programming in Java
Assignment: Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System
Task Number: Task 4
Task Title: Design Classes, Objects and Constructors
Marks: 4
Bloom's Level: K4 – Analyse
Java Version: Java 27

TASK 4 – Design Classes, Objects and Constructors (4 Marks)

---

1. Objective

This task designs the classes identified in Task 1 and decides how their objects are created. It specifies the fields and methods of each class, evaluates default, parameterised and overloaded constructors, uses the this keyword and constructor chaining to avoid repeated code, decides whether a copy constructor is needed (for example when re-issuing a duplicate ticket), and explains how objects and reference variables are held in stack and heap memory. It also compares the chosen overloaded constructors with chaining against a default constructor followed by setter calls and against static factory methods, and analyses the time and space cost of creating n ParkingTicket objects in each approach.

2. Design the Classes Identified in Task 1

The classes and the interface below are those of the Task 1 design. Their names, responsibilities, data types (Task 2) and business rules are unchanged, and no new class has been added. The Task 1 decisions are kept: Vehicle stores the registration number common to all vehicles; ParkingTicket stores the ticket number and the details of the parking session, including the entry time and the exit time; ParkingFacility manages the slots, allocation, release and the active tickets; MonthlyPass manages pass validity; FeeCalculator combines the parking and charging calculations; and Chargeable represents the charging capability of eligible vehicles. Each responsibility is placed in exactly one class.

Table 1 follows on the next page, set in landscape so that all six columns can be read comfortably.

Table 1: Classes and interface of the Campus Smart-Parking System

Class / interface	Purpose and responsibility	Important fields	Data types	Important methods	Justification for the class
Vehicle (abstract class)	Holds the data and behaviour common to every vehicle and declares the abstract hourly rate.	registrationNumber	String (private final)	protected Vehicle(String registrationNumber) String getRegistrationNumber() abstract double ratePerHour() double calculateFee(int minutes) (Task 1)	Avoids repeating the registration number and fee behaviour in every vehicle type; the abstract rate forces each type to supply its own.
TwoWheeler	Vehicle category for two-wheelers.	None of its own	–	TwoWheeler(String registrationNumber) double ratePerHour()	Fee differs by vehicle type, so the type is a class that supplies its own rate.
Car	Vehicle category for cars.	None of its own	–	Car(String registrationNumber) double ratePerHour()	Same reason as TwoWheeler; also the base of ElectricCar.
ElectricCar	A Car that can also be charged.	None of its own	–	ElectricCar(String registrationNumber) double chargingFee(int kWh) (from Chargeable)	Reuses Car and adds only the charging capability.
Chargeable (interface)	Contract for any vehicle that can be charged.	None (an interface has no instance fields and no constructors)	–	double chargingFee(int kWh)	Charging is a capability of EVs only; an interface states it without forcing it on other vehicles.

Class / interface	Purpose and responsibility	Important fields	Data types	Important methods	Justification for the class
ParkingTicket	Records one parking session: who parked where and when, and the fee when it ends.	ticketNumber registrationNumber floor slot entryTime exitTime fee	long String int int LocalDateTime LocalDateTime double	Three constructors (Section 3) boolean isOpen() void close(LocalDateTime exitTime, double fee) getters; toString()	The session must outlive the moment of allocation and carries the unique ticket number; it is closed exactly once.
ParkingFacility	Owns the slots; allocates, releases and reports; issues unique ticket numbers; keeps the active tickets.	slots ticketCounter activeTickets	String[][] long List<ParkingTicket>	ParkingFacility(int[] slotsPerFloor) int[] allocate(String vehicleNo) boolean release(int floor, int slot) availableSlots(); findTicket(String regNo)	One object must own all the slots so that the rule “one vehicle per slot” is enforced in one place.
ParkingUser (abstract class)	Identifies a user and states whether the user can park free.	userId	String	protected ParkingUser(String userId) String getUserId() abstract boolean isEligibleForFreeParking(LocalDate date)	The free-parking rule depends on who the user is, not on the vehicle.
Student	Student user; eligible only with a valid pass.	pass	MonthlyPass	Student(String userId, MonthlyPass pass) isEligibleForFreeParking(LocalDate date)	Own category so the rule can differ from visitors.

Class / interface	Purpose and responsibility	Important fields	Data types	Important methods	Justification for the class
Staff	Staff user; eligible only with a valid pass.	pass	MonthlyPass	Staff(String userId, MonthlyPass pass) isEligibleForFreeParking(LocalDate date)	Own category so the rule can differ from visitors.
Visitor	Visitor user; never eligible for free parking.	None of its own	–	Visitor(String userId) isEligibleForFreeParking(LocalDate date) returns false	Makes “visitors always pay” part of the type.
MonthlyPass	Stores the validity period of a pass and says whether it is valid on a date.	passNumber validFrom validTo	String LocalDate LocalDate	MonthlyPass(String, LocalDate, LocalDate) boolean isValidOn(LocalDate date) getters	An expired pass must give no exemption, so validity needs its own home.
FeeCalculator	Combines the vehicle’s base fee, the pass exemption, the charging fee and any daily maximum into the final amount.	chargingRatePerKWh dailyMaximum	double Double (null = not defined)	FeeCalculator(double chargingRatePerKWh, Double dailyMaximum) totalFee(...) returning parking, charging and total	Pricing policy changes more often than vehicles do, so it is kept in one class.
ParkingApp	Reads input, calls the model and prints results; holds no business rules.	None	–	public static void main(String[] args) input helper methods	Separates the user interface from the model.

Notes on Table 1. (a) VehicleType and the other enumerations of Tasks 2 and 3 are data types used to validate input; they are not extra responsibility classes. (b) ParkingTicket stores the registration number, not a Vehicle reference, as in Task 1. (c) The ticket counter is listed as a field of ParkingFacility as in Task 1; where it is declared is not decided in this task. (d) Methods marked “Task 1” belong to the design but are not needed to demonstrate constructors, so they are not coded in Section 10; Chargeable and ElectricCar.chargingFee are coded in Section 10; an interface has no constructors, so Chargeable declares only the method chargingFee.

3. Constructors in the Parking System

A constructor has the same name as its class and no return type. It runs when an object is created with new and prepares the object for use.

A. Default constructor. A default constructor is a constructor with no parameters. If a class declares no constructor at all, the Java compiler supplies one automatically. The compiler-provided constructor has the same access as the class and an empty body, so every field receives its default value (0 for numbers, false for boolean, null for references). A user-defined no-argument constructor is written explicitly; it can contain initialisation code and can have any access level. As soon as a class declares any constructor, the compiler no longer supplies the default one.

A no-argument constructor is appropriate when an object needs no outside data to be valid. In the parking system this is true of ParkingApp, which holds only main and helper methods, so the compiler-provided constructor is enough:

Code 1: Compiler-provided and user-defined no-argument constructors (illustrative)

```
public class ParkingApp {
    // no constructor written: the compiler supplies  public ParkingApp() { }
    public static void main(String[] args) {
        ParkingApp app = new ParkingApp();   // uses the default constructor
    }
}

// the same class with a user-defined no-argument constructor
public class ParkingApp {
    public ParkingApp() {
        // initialisation that needs no outside data would go here
    }
}
```

It is unsuitable for objects that need mandatory information. A no-argument constructor for ParkingTicket would allow a ticket with ticket number 0, no registration number and no entry time. For this reason ParkingTicket declares only constructors with parameters, so new ParkingTicket() does not compile.

B. Parameterised constructor. A parameterised constructor receives values when the object is created and uses them to initialise the fields. Mandatory information is supplied at construction time because the object is then complete from its first moment, and the constructor can reject bad values before any reference to the object exists, so an invalid object can never be created. The ParkingTicket constructor takes all five items that identify a session:

Code 2: Parameterised constructor of ParkingTicket

```
public ParkingTicket(long ticketNumber, String registrationNumber,
            int floor, int slot, LocalDateTime entryTime) {
    if (ticketNumber <= 0) {
        throw new IllegalArgumentException("ticket number must be positive");
    }
    // ... the other checks (registration number, floor, slot, entry time)
    this.ticketNumber = ticketNumber;
    this.registrationNumber = registrationNumber;
```

```
    this.floor = floor;
    this.slot = slot;
    this.entryTime = entryTime;
}
```

C. Overloaded constructors. Constructor overloading means that a class has several constructors with the same name but different parameter lists, differing in the number, the types or the order of the parameters. The compiler chooses the constructor from the arguments in the call. The return type is not involved, because constructors have none. Several ways of creating the same kind of object are useful when callers hold their data in different forms. ParkingTicket has three constructors: one taking a registration number and an entry time, one taking a Vehicle object (the registration number is read from it), and one that records the current time as the entry time.

Code 3: Overloaded constructors of ParkingTicket (signatures)

```
ParkingTicket(long ticketNumber, String registrationNumber,
    int floor, int slot, LocalDateTime entryTime)    // 1: master constructor
ParkingTicket(long ticketNumber, Vehicle vehicle,
    int floor, int slot, LocalDateTime entryTime)    // 2: from a Vehicle
ParkingTicket(long ticketNumber, String registrationNumber,
    int floor, int slot)                             // 3: entry time = now
```

One caution applies to overloading: a call such as new ParkingTicket(1L, null, 0, 0, entryTime) is ambiguous between constructors 1 and 2 and does not compile, because null fits both String and Vehicle. The caller must pass a typed value.

Table 2: Comparison of constructor types

Constructor type	Syntax	Purpose	Example	Suitable use in the parking system
Default (compiler-provided)	No constructor written; compiler adds ClassName()	Allows new ClassName() when no data is needed	new ParkingApp()	Classes without mandatory data, such as ParkingApp
Default (user-defined no-argument)	public ClassName() { ... }	Same, with explicit initialisation	public ParkingApp() { }	Classes whose fields have sensible defaults; not ParkingTicket
Parameterised	ClassName(Type a, Type b) { ... }	Supplies mandatory data and validates it	new ParkingTicket(1001L, "TN01AB1111", 0, 1, entry)	ParkingTicket, Vehicle subclasses, MonthlyPass

Constructor type	Syntax	Purpose	Example	Suitable use in the parking system
Overloaded	Several constructors with different parameter lists	Offers several valid ways to create the same kind of object	new ParkingTicket(1002L, car, 1, 0, entry)	ParkingTicket, which can be created from a String or a Vehicle

4. Use of this Keyword and Constructor Chaining

The keyword this is a reference to the current object, the object whose constructor or method is running. It has two uses in constructors.

- **Referring to the current object's fields.** When a parameter has the same name as a field, the parameter hides the field inside the constructor. Writing this.floor = floor assigns the parameter floor (on the right) to the field floor of the current object (on the left). Without this, the statement floor = floor would assign the parameter to itself and leave the field unchanged.
- **Calling another constructor of the same class.** The statement this(arguments) makes the constructor delegate to another constructor of the same class that matches the arguments. This is constructor chaining. The shared initialisation and validation are written once, in the master constructor, and the other constructors only prepare their arguments and delegate, so no code is repeated.

The rules for this(...) are as follows. Classically, and in every Java version up to 24, this(...) had to be the first statement of the constructor. Since Java 25 (JEP 513) statements may appear before it, but only statements that do not use the object under construction (no this, no instance fields, no instance methods). To stay valid in all versions, including Java 27, every constructor in this report keeps this(...) as its first statement. A constructor cannot contain both this(...) and super(...), the chain must not call itself in a circle, and the arguments of this(...) may use parameters, constants and static methods but not instance members.

Code 4: Constructor chaining in ParkingTicket (excerpt)

```
// 1: master constructor (Code 2): validates and assigns every field

// 2: overloaded, takes a Vehicle
public ParkingTicket(long ticketNumber, Vehicle vehicle,
                     int floor, int slot, LocalDateTime entryTime) {
    this(ticketNumber, registrationOf(vehicle), floor, slot, entryTime);
}

// 3: overloaded, entry time is the current time to the minute
public ParkingTicket(long ticketNumber, String registrationNumber,
                     int floor, int slot) {
    this(ticketNumber, registrationNumber, floor, slot,
         LocalDateTime.now().truncatedTo(ChronoUnit.MINUTES));
}

private static String registrationOf(Vehicle vehicle) {
```

```
    if (vehicle == null) {
        throw new IllegalArgumentException("vehicle is required");
    }
    return vehicle.getRegistrationNumber();
}
```

Execution order. For the call new ParkingTicket(1003L, "TN03EF3333", 0, 0), the steps are:

1. The compiler selects constructor 3 because the call has four arguments.
2. At run time, memory for the object is allocated and its fields receive default values (0, null, 0.0).
3. The arguments of this(...) are evaluated first, including LocalDateTime.now().truncatedTo(ChronoUnit.MINUTES), so the entry time is known before the master constructor starts.
4. this(...) transfers control to constructor 1. Because it does not start with this(...) or super(...), the compiler inserts a call to the superclass constructor Object() as its first action.
5. Constructor 1 validates all five values and, if they are valid, assigns the fields. If a check fails it throws an exception and the object is never returned to the caller.
6. Control returns to constructor 3, which has no further statements, and the new object's reference is returned to the caller.

For the Vehicle overload (constructor 2) the order is the same, except that step 3 calls the static method registrationOf(vehicle), which checks that the vehicle is not null before constructor 1 runs. The subclasses of Vehicle use the same idea with super: Car(String registrationNumber) begins with super(registrationNumber), which runs the Vehicle constructor and its validation first; ElectricCar calls super(registrationNumber) to reach the Car constructor, which in turn calls Vehicle.

5. Copy Constructor – Is It Needed?

What it is. A copy constructor is a constructor that takes an object of its own class and initialises the new object with the same field values, for example ParkingTicket(ParkingTicket other). Java does not supply one automatically. This differs from the default constructor, which has no parameters, is supplied by the compiler only when no constructor is written, and sets fields to default values instead of copying values from another object. The assignment statement b = a does not copy an object either; it only makes b refer to the same object as a.

When copying is useful. Copying is useful to take a snapshot of an object, or to protect an object from outside changes (defensive copying) when its fields refer to mutable objects.

Is copying a ParkingTicket appropriate? Not as an exact duplicate. The usual reason in the scenario would be re-issuing a ticket, for example when the driver loses it. A plain copy constructor would produce a second ticket with the same ticket number, and that violates the Task 1 rule BR3: a ticket number is unique and is never reused. Two active tickets with one number would also make it impossible to tell which one a driver is presenting.

Decision. The final system does not require a copy constructor in any class. Vehicle, MonthlyPass, ParkingUser and the identity fields of ParkingTicket are immutable (final fields of immutable types such as String, LocalDateTime and LocalDate), so no defensive copy is needed, and a copy of a ParkingFacility would wrongly duplicate its occupancy. ParkingTicket therefore has no plain copy constructor.

If re-issuing is added later. The facility would generate a new unique ticket number from its counter, create a replacement ticket that keeps the registration number, floor, slot and entry time of the original, and withdraw the original from the active tickets. A constructor that takes the original and the new number expresses this safely:

Code 5: Re-issue constructor (illustrative; not part of the submitted source files)

```
private static ParkingTicket requireOriginal(ParkingTicket original) {
    if (original == null) {
        throw new IllegalArgumentException("original ticket is required");
    }
    return original;
}

public ParkingTicket(ParkingTicket original, long newTicketNumber) {
    this(newTicketNumber, requireOriginal(original).registrationNumber,
        original.floor, original.slot, original.entryTime);
    if (newTicketNumber == original.ticketNumber) {
        throw new IllegalArgumentException("a new ticket number is required");
    }
    if (!original.isOpen()) {
        throw new IllegalStateException("a closed ticket cannot be re-issued");
    }
}
```

The constructor handles each case as follows. A null original is rejected by requireOriginal, which is called while the arguments of this(...) are evaluated; arguments are evaluated from left to right, so the null check runs before original.floor, original.slot and original.entryTime are read. The master constructor then validates the new ticket number, which must be positive. The constructor also rejects a new number equal to the original's number, and it rejects a closed original, since a ticket that has already ended must not be re-issued. this(...) is still the first statement of the constructor and delegates to the master constructor, so the field assignments are not repeated. The ticket number is never copied from the original. A constructor cannot know which numbers other tickets already use, so the uniqueness of the new number is guaranteed by ParkingFacility, which takes it from its counter; the constructor only requires a valid number that differs from the original's.

6. Object Creation and Memory Management – Stack and Heap

The Java Virtual Machine keeps objects and local variables in different areas. The description below is a conceptual model; the exact layout, the size of a reference and any optimisations depend on the JVM implementation.

- **Primitive local variables** (for example int floor) and **reference variables** (for example ParkingTicket ticket) are local variables. They live in the frame of the method that declares them, and each method call has its own frame on the stack of the thread that runs it. The frame disappears when the method returns.
- **Objects** are created with new and live in heap memory, which is shared by the whole program. The fields of an object, including its primitive fields, are part of the object on the heap.

- **A reference variable does not contain the object.** It contains a reference to it. The statement ParkingTicket ticket = new ParkingTicket(...) creates the object on the heap and stores a reference to it in the local variable ticket.
- **Several references can refer to one object.** After ParkingTicket sameTicket = ticket, both variables refer to the same object, so a change made through one reference is visible through the other (as the demonstration program in Section 10 shows). A reference whose declared type is Vehicle can refer to an object of a subclass such as Car.
- **Garbage collection.** An object becomes eligible for garbage collection when no live part of the program can reach it through any reference. Setting ticket = null removes only one reference; the object stays reachable through sameTicket. When the last reference is gone, or the frame holding it has ended, the object is eligible. When it is actually reclaimed is decided by the JVM, and the program cannot free it explicitly.

Code 6: Realistic ParkingTicket creation

```
int floor = 0;
int slot = 1;
ParkingTicket ticket = new ParkingTicket(1001L, "TN01AB1111", floor, slot);
ParkingTicket sameTicket = ticket;   // a second reference, no new object
ticket = null;                       // object still reachable via sameTicket
sameTicket = null;                   // now unreachable: eligible for collection
```

Figure 1: Conceptual stack and heap after the first four statements of Code 6

STACK (frame of the running method)		HEAP
floor = 0 (int)		**ParkingTicket object** ticketNumber = 1001 registrationNumber → String object "TN01AB1111" floor = 0 slot = 1 entryTime → LocalDateTime object (current time) exitTime = null fee = 0.0
slot = 1 (int)		
ticket (reference)	→	
sameTicket (reference)	→	

Figure 1 is a conceptual diagram, not a memory dump. The local variables floor and slot hold their values directly on the stack. The variables ticket and sameTicket hold references, drawn as arrows, and both lead to the single ParkingTicket object on the heap, whose fields (including its primitive fields) are stored inside it. The registration number and the entry time are themselves objects, so the ticket holds references to them. No exact sizes in bytes are claimed, because they depend on the JVM.

7. Alternative Object-Creation Approaches

The assignment asks what the solution would be if the objects were created with a default constructor followed by setter calls, or through static factory methods, instead of overloaded constructors with chaining. The three approaches are compared below using the same

ParkingTicket data, and each snippet creates n tickets from arrays regNo, floor, slot and entry that hold the data of the n vehicles.

Approach 1 – Overloaded constructors with constructor chaining (chosen). Every mandatory field is supplied in the constructor call. Validation happens once, in the master constructor, which the other constructors reach through this(...). Either a fully valid object is returned or an exception is thrown.

Code 7: Creating n tickets with constructors

```
ParkingTicket[] tickets = new ParkingTicket[n];
for (int i = 0; i < n; i++) {
    tickets[i] = new ParkingTicket(1001L + i, regNo[i], floor[i], slot[i], entry[i]);
}
```

Approach 2 – Default constructor followed by setters. The object is created empty and each field is assigned by a separate setter call. The fields cannot be final, because they are assigned after construction.

Code 8: Ticket class with setters (illustrative; not part of the submitted source files)

```
public class ParkingTicketWithSetters {
    private long ticketNumber;          // 0 until set
    private String registrationNumber;  // null until set
    private int floor;                  // 0 until set
    private int slot;                   // 0 until set
    private LocalDateTime entryTime;    // null until set

    public ParkingTicketWithSetters() { }

    public void setTicketNumber(long ticketNumber) {
        if (ticketNumber <= 0) {
            throw new IllegalArgumentException("ticket number must be positive");
        }
        this.ticketNumber = ticketNumber;
    }
    // setRegistrationNumber, setFloor, setSlot, setEntryTime in the same way

    public boolean isComplete() {
        return ticketNumber > 0 && registrationNumber != null && entryTime != null;
    }
}

ParkingTicketWithSetters[] tickets = new ParkingTicketWithSetters[n];
for (int i = 0; i < n; i++) {
    ParkingTicketWithSetters t = new ParkingTicketWithSetters();
    t.setTicketNumber(1001L + i);
    t.setRegistrationNumber(regNo[i]);
    t.setFloor(floor[i]);
    t.setSlot(slot[i]);
    t.setEntryTime(entry[i]);
    tickets[i] = t;
}
```

Between new and the last setter the ticket is incomplete: it has ticket number 0, a null registration number and a null entry time. If a setter call is forgotten, the object simply keeps the default value. This is worst for floor and slot, because 0 is itself a valid floor and a valid slot, so a ticket whose setFloor and setSlot were never called looks correct and even passes isComplete(). Each setter can check its own value, but rules that involve several fields (such as an exit time that is not before the entry time) need a separate validation step that every caller has to remember to run, or the checks must be repeated in several setters. The class must also allow the fields to be changed at any time, so a ticket's identity could be altered after it was issued.

Approach 3 – Static factory methods. A static method creates, validates and returns the object. The constructor is made private, so all creation goes through the factory methods, which can have descriptive names.

Code 9: Ticket class with static factory methods (illustrative; not part of the submitted source files)

```
public final class ParkingTicketWithFactory {
    private final long ticketNumber;
    private final String registrationNumber;
    private final int floor;
    private final int slot;
    private final LocalDateTime entryTime;

    private ParkingTicketWithFactory(long ticketNumber, String registrationNumber,
            int floor, int slot, LocalDateTime entryTime) {
        this.ticketNumber = ticketNumber;
        this.registrationNumber = registrationNumber;
        this.floor = floor;
        this.slot = slot;
        this.entryTime = entryTime;
    }

    public static ParkingTicketWithFactory issueAt(long ticketNumber,
            String registrationNumber, int floor, int slot, LocalDateTime entryTime) {
        if (ticketNumber <= 0 || registrationNumber == null
                || registrationNumber.isBlank() || floor < 0 || slot < 0
                || entryTime == null) {
            throw new IllegalArgumentException("invalid ticket data");
        }
        return new ParkingTicketWithFactory(ticketNumber, registrationNumber,
                floor, slot, entryTime);
    }

    public static ParkingTicketWithFactory issueNow(long ticketNumber,
            String registrationNumber, int floor, int slot) {
        return issueAt(ticketNumber, registrationNumber, floor, slot,
                LocalDateTime.now().truncatedTo(ChronoUnit.MINUTES));
    }
}

ParkingTicketWithFactory[] tickets = new ParkingTicketWithFactory[n];
```

```
for (int i = 0; i < n; i++) {
    tickets[i] = ParkingTicketWithFactory.issueAt(
            1001L + i, regNo[i], floor[i], slot[i], entry[i]);
}
```

The factory validates before it creates the object, so, as with constructors, no invalid object is returned, provided that the constructor is private. The names issueAt and issueNow say what each method does, which a pair of overloaded constructors cannot express, and a factory may return an object of a subtype or refuse to create one. Its limitations are that callers must know to use the factory instead of new, a class with only private constructors cannot be extended by subclasses, and the validation is written in the factory, away from the constructor, so another creation route added later could bypass it. A static factory is only a different way of creating the object; it does not by itself change the time or the space needed.

8. Time and Space Complexity Comparison

Let n be the number of ParkingTicket objects created. The analysis assumes a fixed number of fields (seven), a fixed number of setter calls per object (five), a constant-time validation (the registration number has a bounded length, 4 to 12 characters in Task 2, and the other checks are single comparisons), and at most two constructor invocations in any constructor chain.

Table 3: Constructors, setters and static factory methods compared

Criterion	Overloaded constructors with chaining	Default constructor and setters	Static factory methods
Object creation process	new calls the selected constructor, which delegates with this(...) to the master constructor.	new calls the no-argument constructor; one setter call per field follows.	The factory method validates, then calls the private constructor with new, and returns the object.
Validation	Once, in the master constructor, before the object is returned.	In each setter, plus a separate step for multi-field rules that the caller must run.	In the factory method, before the object is created.
Guarantee of a valid object	Strong: no reference exists unless the constructor completed; final fields cannot change later.	None: the object is incomplete until the last setter, and a forgotten setter leaves defaults.	Strong if the constructor is private and every creation goes through the factory.
Time per object	O(1)	O(1)	O(1)
Total time for n objects	O(n)	O(n)	O(n)
Space per object	O(1): a fixed number of fields	O(1): the same fields	O(1): the same fields

Criterion	Overloaded constructors with chaining	Default constructor and setters	Static factory methods
Total space for n objects	O(n)	O(n)	O(n)
Advantages	Mandatory data enforced by the compiler; validation in one place; chaining avoids repeated code; immutable identity fields.	Simple to write; can be created before all data is known.	Descriptive names; validation before creation; flexibility in what is returned.
Limitations	Many parameters become hard to read; callers must supply values in the right order.	Invalid or incomplete objects can exist and be used; fields must stay mutable; more calls per object.	Callers must know to use it; extra static methods to maintain; creation could bypass validation if another route exists.

Time. In every approach the work for one ticket is a fixed number of assignments and constant-time checks, so creating one object takes O(1) time and creating n objects in a loop takes n × O(1) = O(n) time. Constructor chaining adds only constant overhead: a ticket created through constructor 3 runs two constructors instead of one, and because the number of constructors in a chain is fixed the overhead does not depend on n. The setter approach makes five extra method calls per object, which is also a fixed number. The static factory adds one extra method call per object, and it does not reduce the time or the space; it is not faster than a constructor.

Space. Each ticket object holds the same fixed set of fields in all three approaches, so one object needs O(1) space, and n ticket objects held in an array need O(n) space, plus O(n) for the array of references. The registration number and the entry time are shared references to objects that exist anyway. The object holds the same data whichever way it is created, and no pooling or caching of tickets is assumed, since each ticket must be a distinct object with a unique number. The methods use only O(1) auxiliary space while they run.

No benchmark was run and no exact memory size is claimed; the comparison is analytical and the asymptotic results are the same for all three approaches. The approaches differ in correctness guarantees, not in complexity.

9. Design Decision and Justification

Recommendation: overloaded constructors with constructor chaining, with one master constructor that validates and assigns every field. The reasons are:

- **Object validity.** A ParkingTicket can only be created through a constructor that validates its data, so no ticket with an invalid number, a missing registration number or a missing entry time ever exists. With setters such a ticket is possible, and with a factory it is possible only if another creation route exists.
- **Mandatory field initialisation.** The compiler forces the caller to supply every mandatory value; a forgotten value is a compile-time error instead of a silent default such as floor 0.

- **Encapsulation.** The identity fields (ticket number, registration number, floor, slot, entry time) are final and have no setters. The only later changes are the exit time and the fee, which are made together in close(), where the rule that a ticket is closed once and not before it was opened is enforced.
- **Readability.** Three overloads with clearly different parameter lists are easy to read. If many more creation variants were needed, named static factory methods that delegate to the same constructor would be a sensible addition.
- **Maintainability.** Validation is written once, in the master constructor, and the other constructors only delegate to it, so a changed rule is edited in one place.
- **Future extension.** A new way of creating a ticket, such as the re-issue constructor of Section 5, is another overload that delegates to the master constructor, and none of the existing code changes.

The same strategy suits the other classes: Vehicle and its subclasses, and MonthlyPass, each have mandatory data that must be valid from the start and is not changed afterwards, so each has a parameterised constructor that validates it. ParkingApp has no mandatory data and uses the compiler-provided default constructor. ParkingFacility keeps the constructor of Task 3 that builds the slot structure from the capacities of the floors.

10. Java Class Definitions

The files below demonstrate the Task 4 concepts: fields, parameterised and overloaded constructors, this, constructor chaining and object creation with references. They do not repeat the whole parking application; ParkingFacility is the Task 3 class and is not repeated here, and no package declaration is used. The code does not use any feature that is specific to a Java version, so it can be compiled with Java 27.

Table 4: Source files of Task 4

File	Purpose
Vehicle.java	Abstract class with the registration number and a validating constructor.
Chargeable.java	Interface that declares double chargingFee(int kWh).
TwoWheeler.java, Car.java	Vehicle categories; each constructor calls super(registrationNumber). The rates are placeholders and not assignment requirements.
ElectricCar.java	Extends Car and implements Chargeable; the constructor calls super(registrationNumber), and chargingFee(int kWh) rejects a negative kWh and returns kWh × 12.0, an illustrative rate and not an assignment requirement.
MonthlyPass.java	Pass number and validity dates with a validating constructor and isValidOn().
ParkingTicket.java	The ticket with a parameterised constructor, two overloads and chaining with this(...).
Task4Demo.java	Small demonstration of object creation, reference sharing and rejected invalid objects.

No separate source file is provided for the Student, Staff, Visitor or FeeCalculator classes, because their constructors follow the same pattern and are specified in Table 1.

Code 10: Vehicle.java

```
public abstract class Vehicle {
    private final String registrationNumber;

    protected Vehicle(String registrationNumber) {
        if (registrationNumber == null || registrationNumber.isBlank()) {
            throw new IllegalArgumentException(
                    "registration number is required");
        }
        this.registrationNumber = registrationNumber;
    }

    public String getRegistrationNumber() {
        return registrationNumber;
    }

    // hourly rate of the vehicle type (placeholders in the subclasses)
    public abstract double ratePerHour();
}
```

Code 11: TwoWheeler.java

```
public class TwoWheeler extends Vehicle {
    public TwoWheeler(String registrationNumber) {
        super(registrationNumber);
    }

    @Override
    public double ratePerHour() {
        return 10.0;   // placeholder rate, not an assignment requirement
    }
}
```

Code 12: Car.java

```
public class Car extends Vehicle {
    public Car(String registrationNumber) {
        super(registrationNumber);
    }

    @Override
    public double ratePerHour() {
        return 30.0;   // placeholder rate, not an assignment requirement
    }
}
```

Code 13: Chargeable.java

```
public interface Chargeable {
    // charging fee for the units (kWh) supplied to the vehicle
    double chargingFee(int kWh);
}
```

Code 14: ElectricCar.java

```
public class ElectricCar extends Car implements Chargeable {
    public ElectricCar(String registrationNumber) {
        super(registrationNumber);
    }

    @Override
    public double chargingFee(int kWh) {
        if (kWh < 0) {
            throw new IllegalArgumentException("kWh must not be negative");
        }
        return kWh * 12.0;   // illustrative rate, not an assignment requirement
    }
}
```

Code 15: MonthlyPass.java

```
import java.time.LocalDate;

public class MonthlyPass {
    private final String passNumber;
    private final LocalDate validFrom;
    private final LocalDate validTo;

    public MonthlyPass(String passNumber, LocalDate validFrom,
                       LocalDate validTo) {
        if (passNumber == null || passNumber.isBlank()) {
            throw new IllegalArgumentException("pass number is required");
        }
        if (validFrom == null || validTo == null) {
            throw new IllegalArgumentException("validity dates are required");
        }
        if (validTo.isBefore(validFrom)) {
            throw new IllegalArgumentException(
                    "valid-to date must not be before valid-from date");
        }
        this.passNumber = passNumber;
        this.validFrom = validFrom;
        this.validTo = validTo;
    }

    public boolean isValidOn(LocalDate date) {
        return date != null
```

```
                && !date.isBefore(validFrom)
                && !date.isAfter(validTo);
    }

    public String getPassNumber() {
        return passNumber;
    }

    public LocalDate getValidFrom() {
        return validFrom;
    }

    public LocalDate getValidTo() {
        return validTo;
    }
}
```

Code 16: ParkingTicket.java

```
import java.time.LocalDateTime;
import java.time.temporal.ChronoUnit;

public class ParkingTicket {
    private final long ticketNumber;
    private final String registrationNumber;
    private final int floor;
    private final int slot;
    private final LocalDateTime entryTime;
    private LocalDateTime exitTime;   // null while the ticket is open
    private double fee;               // set when the ticket is closed

    // Parameterised constructor: validates and initialises every field
    public ParkingTicket(long ticketNumber, String registrationNumber,
                         int floor, int slot, LocalDateTime entryTime) {
        if (ticketNumber <= 0) {
            throw new IllegalArgumentException(
                    "ticket number must be positive");
        }
        if (registrationNumber == null || registrationNumber.isBlank()) {
            throw new IllegalArgumentException(
                    "registration number is required");
        }
        if (floor < 0 || slot < 0) {
            throw new IllegalArgumentException(
                    "floor and slot must not be negative");
        }
        if (entryTime == null) {
            throw new IllegalArgumentException("entry time is required");
        }
        this.ticketNumber = ticketNumber;
        this.registrationNumber = registrationNumber;
```

```
        this.floor = floor;
        this.slot = slot;
        this.entryTime = entryTime;
    }

    // Overloaded constructor: takes a Vehicle, not a registration number
    public ParkingTicket(long ticketNumber, Vehicle vehicle,
                         int floor, int slot, LocalDateTime entryTime) {
        this(ticketNumber, registrationOf(vehicle), floor, slot, entryTime);
    }

    // Overloaded constructor: entry time is the current time (minutes)
    public ParkingTicket(long ticketNumber, String registrationNumber,
                         int floor, int slot) {
        this(ticketNumber, registrationNumber, floor, slot,
                LocalDateTime.now().truncatedTo(ChronoUnit.MINUTES));
    }

    private static String registrationOf(Vehicle vehicle) {
        if (vehicle == null) {
            throw new IllegalArgumentException("vehicle is required");
        }
        return vehicle.getRegistrationNumber();
    }

    public boolean isOpen() {
        return exitTime == null;
    }

    public void close(LocalDateTime exitTime, double fee) {
        if (!isOpen()) {
            throw new IllegalStateException("ticket is already closed");
        }
        if (exitTime == null || exitTime.isBefore(entryTime)) {
            throw new IllegalArgumentException(
                    "exit time must not be before entry time");
        }
        if (fee < 0) {
            throw new IllegalArgumentException("fee must not be negative");
        }
        this.exitTime = exitTime;
        this.fee = fee;
    }

    public long getTicketNumber() {
        return ticketNumber;
    }

    public String getRegistrationNumber() {
        return registrationNumber;
    }
```

```
    public int getFloor() {
        return floor;
    }

    public int getSlot() {
        return slot;
    }

    public LocalDateTime getEntryTime() {
        return entryTime;
    }

    public LocalDateTime getExitTime() {
        return exitTime;
    }

    public double getFee() {
        return fee;
    }

    @Override
    public String toString() {
        return "Ticket " + ticketNumber + " " + registrationNumber
                + " floor " + floor + " slot " + slot + " entry " + entryTime
                + (isOpen() ? " (open)"
                        : " exit " + exitTime + " fee " + fee);
    }
}
```

Code 17: Task4Demo.java

```
import java.time.LocalDate;
import java.time.LocalDateTime;

public class Task4Demo {
    public static void main(String[] args) {
        // 1. parameterised constructor
        ParkingTicket t1 = new ParkingTicket(1001L, "TN01AB1111", 0, 1,
                LocalDateTime.of(2026, 11, 2, 9, 15));
        System.out.println(t1);

        // 2. overloaded constructor taking a Vehicle (chains to the first)
        Vehicle car = new Car("TN02CD2222");
        ParkingTicket t2 = new ParkingTicket(1002L, car, 1, 0,
                LocalDateTime.of(2026, 11, 2, 9, 30));
        System.out.println(t2);

        // 3. overloaded constructor that uses the current time
        ParkingTicket t3 = new ParkingTicket(1003L, "TN03EF3333", 0, 0);
        System.out.println("entry time set: " + (t3.getEntryTime() != null));
```

```
        // 4. two references to the same object
        ParkingTicket alias = t1;
        System.out.println("same object: " + (alias == t1));
        t1.close(LocalDateTime.of(2026, 11, 2, 10, 16), 60.0);
        System.out.println("open through alias: " + alias.isOpen());
        System.out.println(alias);

        // 5. invalid objects are rejected during construction
        try {
            new ParkingTicket(0L, "TN04GH4444", 0, 0,
                    LocalDateTime.of(2026, 11, 2, 9, 0));
        } catch (IllegalArgumentException e) {
            System.out.println("rejected: " + e.getMessage());
        }

        // 6. constructors in the Vehicle classes and in MonthlyPass
        Vehicle bike = new TwoWheeler("TN05IJ5555");
        Vehicle ev = new ElectricCar("TN06KL6666");
        System.out.println(bike.getRegistrationNumber() + " "
                + bike.ratePerHour() + " | "
                + ev.getRegistrationNumber() + " " + ev.ratePerHour());

        MonthlyPass pass = new MonthlyPass("P-2026-001",
                LocalDate.of(2026, 11, 1), LocalDate.of(2026, 11, 30));
        System.out.println(pass.isValidOn(LocalDate.of(2026, 11, 2)) + " "
                + pass.isValidOn(LocalDate.of(2026, 12, 1)));
        try {
            new MonthlyPass("P-2026-002",
                    LocalDate.of(2026, 11, 30), LocalDate.of(2026, 11, 1));
        } catch (IllegalArgumentException e) {
            System.out.println("rejected: " + e.getMessage());
        }
    }
}
```

Compiling and running. The files depend on each other, so they are compiled together. Save all eight files in the Task04 folder, open a terminal in that folder (Section 9 of the Task 3 report explains how) and run the two commands below. The first compiles every .java file in the folder, which here means exactly the eight files of Table 4.

```
javac *.java
java Task4Demo
```

Expected output of Task4Demo. The output follows from the code; the third line depends only on the entry time being set, not on its value, so the program prints the same ten lines every time it runs.

Table 5: Expected output of Task4Demo and the reason for each line

No.	Expected output	Why it occurs
1	Ticket 1001 TN01AB1111 floor 0 slot 1 entry 2026-11-02T09:15 (open)	The parameterised constructor created the ticket; exitTime is null, so isOpen() is true.
2	Ticket 1002 TN02CD2222 floor 1 slot 0 entry 2026-11-02T09:30 (open)	The Vehicle overload chained to the master constructor; the registration number came from the Car object.
3	entry time set: true	The four-parameter overload took the entry time from the clock; only its presence is printed.
4	same object: true	alias = t1 copied the reference, so == finds one object.
5	open through alias: false	close() was called through t1; the change is visible through alias because there is one object.
6	Ticket 1001 TN01AB1111 floor 0 slot 1 entry 2026-11-02T09:15 exit 2026-11-02T10:16 fee 60.0	The same ticket after closing, printed through alias.
7	rejected: ticket number must be positive	The constructor threw an exception for ticket number 0, so no object was created.
8	TN05IJ5555 10.0 | TN06KL6666 30.0	The subclass constructors called super(...); ElectricCar inherits the rate of Car (placeholder values).
9	true false	The pass is valid on 2 November 2026 and not on 1 December 2026.
10	rejected: valid-to date must not be before valid-from date	The MonthlyPass constructor rejected a validity period that ends before it starts.

11. Code Explanation

- **Constructor invocation.** new ParkingTicket(1001L, "TN01AB1111", 0, 1, LocalDateTime.of(2026, 11, 2, 9, 15)) selects the master constructor because it has five arguments of types long, String, int, int and LocalDateTime. new ParkingTicket(1002L, car, 1, 0, ...) selects the Vehicle overload because the second argument is a Vehicle, and new ParkingTicket(1003L, "TN03EF3333", 0, 0) selects the four-parameter overload.
- **Parameters initialise fields.** In the master constructor, each assignment such as this.ticketNumber = ticketNumber stores the parameter in the field of the new object. The fields ticketNumber, registrationNumber, floor, slot and entryTime are final, so they are assigned exactly once, and only after all checks have passed.
- **Role of this.** On the left of an assignment, this.floor is the field of the current object; on the right, floor is the constructor parameter. In close(), this.exitTime = exitTime and this.fee = fee do the same. In the two overloads, this(...) is the first statement and delegates to the master constructor.

- **Constructor chaining.** The Vehicle overload calls this(ticketNumber, registrationOf(vehicle), floor, slot, entryTime); the clock overload calls this(ticketNumber, registrationNumber, floor, slot, LocalDateTime.now().truncatedTo(ChronoUnit.MINUTES)). Both reach the same validation and assignments, so nothing is repeated. registrationOf is static because the arguments of this(...) cannot call instance methods. The subclass constructors use super(registrationNumber) as their first statement to run the Vehicle constructor.
- **Chargeable and ElectricCar.** Chargeable is an interface, so it has no constructor and declares only chargingFee(int kWh). ElectricCar extends Car and implements Chargeable: its constructor calls super(registrationNumber), and chargingFee throws IllegalArgumentException for a negative kWh and otherwise returns kWh × 12.0, an illustrative rate. Task4Demo does not call chargingFee, so its expected output is not affected.
- **References and objects.** In Task4Demo, t1 holds a reference to the ParkingTicket object on the heap. ParkingTicket alias = t1 copies the reference and creates no new object, which lines 4 to 6 of the expected output show. Vehicle car = new Car("TN02CD2222") shows a Vehicle reference that refers to a Car object.
- **Preventing invalid objects.** The master constructor throws IllegalArgumentException when the ticket number is not positive, the registration number is null or blank, the floor or slot is negative, or the entry time is null; the Vehicle and MonthlyPass constructors do the same for their data. An exception stops the construction, so the caller never receives a reference to an invalid object, and there is no no-argument constructor that could create an empty one. The constructor cannot check the upper limit of the floor and slot because the ticket does not know the size of the facility; ParkingFacility chooses them.

12. Result

The classes of Task 1 were designed with fields, data types and methods, and each class and interface was justified. Default, parameterised and overloaded constructors were evaluated, and ParkingTicket was given a validating parameterised constructor and two overloads. The keyword this and constructor chaining with this(...) were demonstrated, and the execution order of the constructors was explained. The need for a copy constructor was evaluated and the decision was that none is required, because copying a ticket as it is would duplicate a ticket number; a re-issued ticket needs a new unique number. Stack and heap memory, reference variables and garbage-collection eligibility were explained with a conceptual diagram. Overloaded constructors with chaining were compared with default constructor plus setters and with static factory methods. For creating n ParkingTicket objects, each approach takes O(1) time per object, O(n) total time and O(n) total space, and the approaches differ in the guarantee of a valid object, not in complexity.

Execution record (to be completed by the student after running the program):

Check	Done
Java 27 installed	[ ]
All eight files compiled together with javac *.java	[ ]
Program executed with java Task4Demo	[ ]
Actual output matched the expected output in Table 5	[ ]
Screenshot saved as output.png in the Task04 folder	[ ]